

In the Neighborhood: Revisiting kNN as a tool for Movie Recommendation Systems

Luis Barriga
TSYS College of Computer Science
Columbus State University
Columbus, GA USA
barriga_luis@columbusstate.edu

Ashley Bedford
TSYS College of Computer Science
Columbus State University
Columbus, GA USA
bedford_ashley@columbusstate.edu

Aida Hashemlou
TSYS College of Computer Science
Columbus State University
Columbus, GA USA
hashemlou_ada@columbusstate.edu

Abstract— Recommender systems are used in a broad variety of domains in our lives. From item recommendations on e-commerce sites such as Amazon to song recommendations by Spotify or Apple music, this type of machine learning algorithm has found its way to many of our everyday interactions with technology. Perhaps the most ubiquitous among these are movie recommendation systems, such as the ones used by Netflix, Amazon Video, and Hulu. After all, even before a child has a bank account to shop online or their own music subscription, they’ve likely interacted with a streaming service that carries their favorite cartoons, TV shows, and movies. Similarly ubiquitous among recommender systems algorithms is the kNN algorithm. While straightforward in its approach, it remains a reliable benchmark in applications where similarity calculations are involved, as that is the core of the algorithm’s logic. This inspired the authors to revisit this tried-and-true approach in a well-researched domain so that we may better understand the strengths and limitations of the approach and offer our own insights.

Keywords— kNN, Recommender Systems, Popularity Algorithm, Pearson Correlation Based Similarity, IMDB, Movies

I. INTRODUCTION

We live in an age where interactions with recommender systems are commonplace. Whether we are shopping online or consuming media, providers want to make sure their offerings on display are as relevant as possible and as likely to be enjoyed by their customers as competition is high and attention spans are short. One of the primary domains where we have seen these systems develop has been in the streaming industry, where titans such as Netflix and Amazon Video have made it their business to get to know our tastes and refine algorithms that keep their recommendations suitable, relevant, and interesting.

Of the various methods deployed, the k-nearest neighbors algorithm, which is an important and versatile foundational technique in the field of machine learning, has withstood the test of time as it is still relevant in spite of its age and simplicity. Its origins can be traced to the paper: “Nonparametric Discrimination: Consistency Properties” by Evelyn Fix and J.L. Hodges [1]. The original purpose of the algorithm is described in the paper as attempting to solve what it calls “the discrimination problem,” which consists of identifying whether a given variable with many dimensions belongs to one of two distributions.

At its core, the methods described in the paper handle a classification problem from a probabilistic approach, simply put what is the probability that we make a right or wrong classification decision with the assumptions that we make. It also introduces the concepts of “neighborhoods” that can be shrunk to the right size to minimize this error. This basically is the idea that evolves into the “k” in k-nearest neighbors (kNN), although that was not directly described this way in the paper.

How is it then that an algorithm originally designed for classification for two labels becomes useful to us in a recommender system? The paper “Application of Dimensionality Reduction in Recommender System - A Case Study” gives us some insight. Though the practice was already widespread at the time the paper was published, it introduced a key innovation.

One of the techniques in the early days of e-commerce was to build “neighborhoods” of similar customers and pull recommendations out of those clusters. At its core, it is the k-nearest neighbors approach using Pearson coefficients or cosine similarity for its comparison. When we think of customers as composed of features that consist of each product they’ve given a rating to, we realize that there are many products and most customers only consume and rate a fraction of them. In essence, they are high-dimensional and high-sparsity [2].

Not only does the paper by Sarwar, et al. give us some insight into how kNN transitioned into being good for recommender systems, it also proposes the innovation of dimensionality reduction via Singular Value Decomposition. The idea consists of using linear algebra to express a high dimensional matrix as the product of multiple low dimensional matrices. Though implementing SVD fell outside the scope of our project, it is an important technique that helps us understand how larger scale projects can be tackled.

The kNN algorithm presents advantages over some rules-based approaches as no knowledge of the preferences of various demographics of the users are necessary. A traditional rules-based approach would have to rely on an expert to provide deep knowledge and current understanding of reliable correlations between demographics and preferences. Additionally, questionnaires or mined data that would give us hints as to what the users preferences might be in order to inform those rules would mean that the data we had on hand would not be very

helpful. Relying on more generalized rules, such as popularity metrics, is a straightforward approach that does not rely on metrics, but perhaps misses an opportunity to tailor a recommendation to the specific user.

Finally, with a rules based approach, particularly a fuzzy one, we would need many rules to cover all the genres and subgenres that could be difficult to test. In our experience, a high number of rules can create sparsity problems that aren't easy to spot at first and are only identified much later. The appeal of a simple, time-tested, machine learning algorithm to solve the core recommendation problem served as the deciding factor of choosing this direction. As we'll see, however, our conversation around rules-based approaches informed some ideas that would help us address the limitations of kNN with some light hybridization.

II. DATASET DESCRIPTION

A. The MovieLens Datasets

MovieLens, run by GroupLens, has collected user preferences related to movies for many years and offers this data for educational use through their website [3]. Additional versions of this data are also mirrored in websites such as Kaggle.

The data is contained within three comma separated value (.csv) files:

- movies.csv contains a "movieId" column, consisting of numerical indices for each movie in the database, the movie "title" column and a "genre" column for the movie.
- ratings.csv contains "userId," which is a numerical index for each user, "movieId" which is the id of the rated movie, "rating" which is the 1-5 rating given by the user, and "timestamp" which contains the time the rating was given. The "timestamp" feature was not used in our models.
- tags.csv contains user-generated tags that can vary, from subgenre information, to actors, directors, or content tags. Due to a high number of uniquely titled tags, we chose to not use this information in our approach.

B. Choosing an Appropriate Dataset Size

Various sizes of this dataset are offered on the MovieLens site. We initially examined the 25m (25,000,000 rating) version of the data and found that it presented problems in being able to load that much data into the high dimensionality matrices required by kNN, iterate quickly, and evaluate our recommender system. If we were to implement SVD, it would perhaps be possible for us to work with this dataset, though iteration time would have still been considerable.

A second attempt to run the data was made against the latest version of "small" dataset (100,000 ratings) and were able to quickly process the data and load it as a combined matrix into our recommender. We found that this dataset was more suitable due to its ability to run through quick iterations and complete an evaluation portion for our project.

III. PROPOSED METHOD

A. User-based vs content-based filtering

One of the first decisions that must be made when applying kNN to a recommender system is whether one wants kNN to offer similar items directly, also known as content-based or item-based filtering, or whether one wants to find similar users and build recommendations based on those users' preferences, also known as user-based filtering [4].

			
	★★★★★	★★	★★★★★
	★★★★	★★★★★	★★★★
	★★★★★	★★	★★★★★

Fig. 1: User-based Filtering Data Table Example

In user-based filtering, each movie rating can be used as the features for each user and find similar users. In this example, viewing the user icons from top to bottom, the first and third users are similar because they've rated movies similarly (1).

			
	★★★★★	★★★★	★★★★★
	★★	★★★★★	★★
	★★★★★	★★★★	★★★★★

Fig. 2: Content-based Filtering Data Table Example

In content-based filtering, each user rating can be used as the features for each movie and find similar movies. In this example, viewing the movie icons from top to bottom, the first and third movies are similar movies because they have been rated similarly (2).

Since the ultimate goal is to recommend a movie to a user, implementing user-based filtering involves an additional step in order to build recommendations based on what those users also liked. Additionally, the user-based filtering approach has a "cold-start" problem in that we cannot make reliable recommendations until the user has made a few predictions. For these reasons, we chose to implement an item-based filtering technique.

Content-based filtering has its own cold-start problem of new movies not being recommended before users have rated them but that is less applicable to our exercise as we won't be adding completely new movies to our recommender. It also cannot provide an initial recommendation until the user has rated a movie, a problem which is solved by populating an initial list.

B. Data Pre-processing

To use the Movielens 100K movie and rating data with our KNN model, the csv data needs to be transformed into a 2D CSR Matrix, where the rows represent movies and the columns represent users. This matrix is then used as the input for our KNN Model. The CSR Matrix is a compressed and memory-efficient format that is ideal for representing sparse matrices, where most of the entries are zero. In our recommendation system, the CSR matrix is used to represent user-item interactions, and the KNN algorithm can efficiently compute similarity measures between users or items by only considering the non-zero entries in the matrix for those specific users or items. This saves time when working with large datasets, as we don't need to iterate over every possible pair of items. Furthermore, using the CSR Matrix format helps reduce the memory requirements for storing the user-item interactions, making it possible to work with larger datasets even with limited resources as noted by Aioli [7].

A challenge of using this CSR matrix approach was discovering ways that our system would handle nan values. Using the movies that users have as nan ratings in our matrix would halt our ability to properly evaluate the KNN model, since nan is not a float value similar to the ratings data type. Since our dataset did not have a user who inputted a rating of zero, our program can fill in all the movies that were not rated by users as nan with the value of zero and ignore them from being evaluated in the recommendation system all together. Ignoring all zeroes will prevent the model from skewing the results drastically towards zero. However, there is a limitation to this approach if we would like to include non-rated movies in our evaluation. Changing Nan values to the value of zero would make the movie appear closer to unfavorably rated movies. Therefore, imputing an "average" rating such as 2.5 would be an improvement to fit that approach.

C. kNN Distance Metrics

Different variants of kNN exist, each with their own set of metrics to measure the similarity of the sampled point and its nearest neighbors. Some of the similarities that we chose to analyze are: Euclidean distance, Cosine Similarity, Minkowski distance, and Manhattan distance (also known as "City-block").

Intuitively, this process of measuring similarity might be thought of measuring how far off are two points from each other in space, that is the smallest line signifying the distance that exists between two points. This is precisely how the Euclidean distance metric works. If we consider the various users to be the dimensions of each movie and the ratings to be the values of those dimensions, the difference between the two points in multi-dimensional space corresponds to the Euclidean distance. The Euclidean distance, $d(p, q)$, formula is as follows:

$$d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2} \quad (1)$$

where,

- (p, q) as p and q , = two points in Euclidean n-space
- q_i and p_i = Euclidean vectors, starting from the initial point of the space
- n = the Euclidean n-space.

In turn, cosine similarity measures the similarity in the angles that the vectors form and might be better suited for high-dimensional matrices, which we will attempt to gauge in our evaluation. The Cosine Similarity, $\cos(x, y)$, formula is as follows:

$$\cos(x, y) = \frac{x \cdot y}{\|x\| * \|y\|} \quad (2)$$

where,

- $x \cdot y$ = the dot product of x and y
- $\|x\| * \|y\|$ = length of the two vectors 'x' and 'y' multiplied together

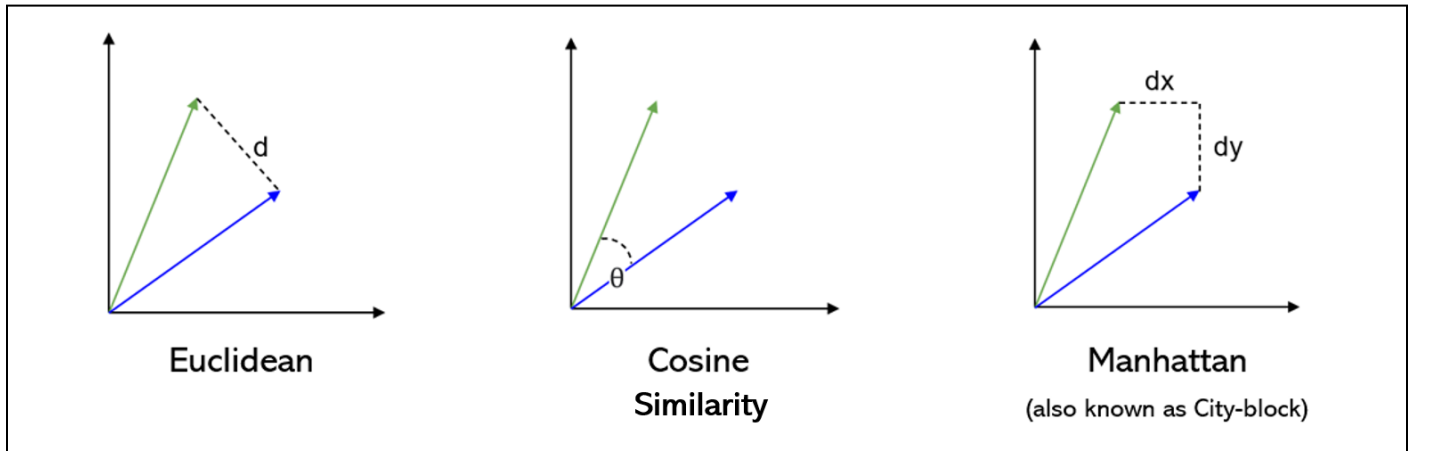


Fig. 3: Different Distance Calculations

Other variations of the distance metrics offer additional permutations to the calculations. For example one can think of Manhattan “City-block” as a version of Euclidean that is not allowed to measure diagonal measurements and must add each vector’s difference in magnitude to the calculation. Hence the term city-block represents traversing two lengths of a block on a grid instead of diagonally. An example of this is shown above (3). The Manhattan, $d(x, y)$, distance formula is as follows:

$$d(x, y) = |x_1 - x_2| + |y_1 - y_2| \quad (3)$$

where,

- (x_1, y_1) = the first pair of coordinates
- (x_2, y_2) = the second pair of coordinates

The Minkowski distance metric consists of what can be considered as a generalization of the Euclidean and Manhattan distances. The Minkowski, $d(x_i, y_i)$, distance formula is as follows:

$$d(x_i, y_i) = (\sum_{i=1}^n |x_i - y_i|^p)^{\frac{1}{p}} \quad (4)$$

where,

- (x_i, y_i) = a pair of coordinates
- p = any real value that is typically set to 1 or 2, where $p = 1$ is the same as the Manhattan distance and $p = 2$ is the same as the Euclidean distance

D. Project Implementation Method Selection

From the start of the project, we set out to pursue a dual approach to our exploration. We wanted to create a notebook where we could utilize the data to run evaluation metrics against each distance metric and create a GUI-based app so that we could simulate the user experience with our movie recommender system. Implementing a hybrid system would allow our model to be supplemented with advantages where kNN may fall short.

Creating a kNN model has highlighted the major challenges that our team has faced in which the hybrid system aided us in solving. Handling negative feedback from the user in the form of a rating less than three conflicted with the purpose of using kNN. The kNN model is used to return similar movies to the users liked movies in order to return a recommendation. When a user rates a movie negatively, the kNN movie will not consider the rating a poor and return similar movies to it. This is not the intended output of our model. Therefore, a popularity-based algorithm was introduced to return a list of recommended movies that are the most popular and highly rated by all users. This is a positive result that has a higher likelihood of a user liking and rating a movie.

Another example of a challenge that we faced which was solved by our hybrid model was handling movie recommendations for users that have not rated anything yet. The popularity-based algorithm and rules proved useful here as they provided recommendations based highly rated movies overall. This approach provided a solution for handling the cold-start

problem, thus addressing some of the shortcomings of kNN as a recommendation method.

E. Rules-based Considerations

Popularity rule-based recommendation systems have become increasingly widespread in recent years due to their simplicity and ease of implementation. One of the key benefits of popularity-based recommendation systems is their ability to provide relevant recommendations to new users. Since these systems do not require any prior knowledge of a user's preferences, they can easily provide recommendations based on the popularity of items. This makes them particularly useful for situations where a large number of data is available and new users may not have established a well-defined favorites list yet.

The popularity score is calculated by counting the number of times an item has been chosen or viewed by users. A simple formula for calculating the Popularity Score (PS) is as follows:

$$PS = \frac{c}{N} \quad (5)$$

where,

- c = the number of times an item was chosen
- N = total number of choices

The popularity score can be normalized to fall within a range of zero to one by dividing it by the maximum popularity score. This normalization process ensures that items with different levels of popularity can be compared fairly. The formula for calculating the normalized popularity score is as follows [5]:

$$PS_n = \frac{PS}{\max(PS)} \quad (6)$$

where,

- PS = the item’s popularity score
- $\max(PS)$ = total maximum popularity score

Once the items are ranked by their popularity scores, the top-ranked items can be recommended to users. However, this approach has some limitations. Firstly, it only recommends popular items, which may not necessarily be relevant to the specific user's interests. Secondly, it does not consider the user's preferences. To address these limitations, popularity-based filtering can be combined with other filtering techniques, such as content-based filtering, to create a hybrid recommender system.

There are many filtering techniques available to be used in a recommender system. One of the most popular approaches for building hybrid recommender systems is the weighted average technique. In this technique, recommendations from multiple filtering techniques are combined using a weighted average. The weights for each technique are determined based on their performance in predicting user preferences.

The formula for calculating the weighted average is as follows [6]:

$$WA = \sum(w_n * s_n) \quad (7)$$

where,

- w_n = the weights for each filtering technique
- s_n = the scores for each recommendation from each technique

The IMDB weighted average formula is a well-established and respected measure of movie quality, which is being used in many movie streaming services as well as many websites such as Rotten Tomatoes and Metacritic.

We used this formula in our project to generate an initial list of the recommendations to overcome the cold-start problem described earlier in the paper. Also in cooperation with some other rules, the weighted rule-based recommendation part of the system is utilized to create a highly sophisticated tool for providing personalized movie recommendations to users based on popularity upon user demand.

This formula takes into account both the number of votes and the average rating of a movie to calculate a weighted rating score. This score provides an objective measure of a movie's quality, which our system can use to make informed recommendations.

The weighted average formula used by IMDb is expressed as follows [9]:

$$WR = \left[\frac{v}{(c+m)} \right] + \left[\frac{m \cdot C}{(v+m)} \right] \quad (8)$$

where:

v = number of votes received by the movie

m = minimum number of votes required to be listed in the top recommendations list

c = the mean vote across the whole report

C = the mean vote across the movie

The formula consists of two parts. The first part of the formula calculates the average rating of the movie based on the number of votes it has received. The second part of the formula adds a "weighted" component to the average rating. This second part of the formula ensures that movies with fewer votes are not over-represented in the overall rating.

The minimum number of votes required to be listed in our system is currently 10. This means that a movie must have received at least 10 votes to be included in the top ten list. The reason for this is to ensure that the top-rated movies are truly representative of the views of enough number of people.

The mean vote across the whole report is calculated by taking the average of all the ratings given to all the movies on the dataset. The mean vote across the movie is calculated by taking the average rating of the movie.

As soon as the user rates a movie and picks the popularity method to submit their vote into, the system takes note of the selected genre(s) and generates the next list of recommendations based on that. With each subsequent rating, the system assigns scores from +2 to -2 to the selected genres based on the user's rating. Next, it picks the highest and lowest-scored genres and

matches them together to make the prioritization rule in the recommendation process. This allows the system to continuously adapt to the user's preferences and provide more relevant and versatile recommendations over time.

Another feature of the system is that it removes any movie that the user rates from the recommendation list. This ensures that the user is always presented with new and relevant recommendations. Additionally, the system resets the genre scores if they reach +20 or -20, preventing the system from becoming too focused on a particular genre and providing a more balanced set of recommendations.

The system also shuffles the final list of recommendations. This helps to prevent the system from becoming too predictable and ensures that the user is presented with a diverse set of recommendations. By shuffling the list, the system also encourages the user to explore new genres and discover movies they may not have otherwise considered.

Another implementation of the weighted popularity system is when the user rates a movie with scores 1 or 2. Since the KNN system lacks the ability to recognize the non-favorite titles, the system automatically switches to a weighted popular list of items to cover this flaw.

The final algorithm pseudocode that has been integrated into our system is as follows:

- User submits their vote for a title
- The system takes note of the selected genre(s)
- It removes the selected movie from the recommendation list
- The system assigns scores from +2 to -2 to the selected genres based on the user's rating
- It picks the highest and lowest-scored genres and matches them together to make the prioritization rule
- Top ten recommended list is calculated using the formula
- The system shuffles the final list of recommendations before presenting to user
- The system resets the genre scores if they reach +20 or -20.

By assigning scores to genres based on user ratings, the system is able to identify patterns in the user's preferences and adjust its recommendations accordingly. This allows our system to learn from the user's ratings and provide increasingly accurate recommendations over time and ensure that the user is presented with movies they are likely to enjoy.

F. Experimental Data and Results

Finding an evaluation approach for our item-based recommendation system was difficult due to the lack of examples and documentation to present methods for finding an approach to effectively evaluate our model's performance. Since we are recommending new movies to a user using previously watched movies, how do we know our recommendations were accurate? That information will not be known with 100% certainty until the user watches and rates the movie, therefore a

model was created to make a prediction on how well it believes that the user will like our recommendations. Our chosen approach was a modified Pearson Correlation Based Similarity with a fixed rating threshold algorithm. The Pearson Correlation Based Similarity algorithm is below:

$$w_{i,j} = \frac{\sum_{u \in U} (r_{u,i} - \bar{r}_i)(r_{u,j} - \bar{r}_j)}{\sqrt{\sum_{u \in U} (r_{u,i} - \bar{r}_i)^2} \sqrt{\sum_{u \in U} (r_{u,j} - \bar{r}_j)^2}} \quad (9)$$

where:

$r_{u,i}$ = the user's rating for a movie

$\bar{r}_i$ = the average rating for the movie across all users

Using the figure provided by Su and Khoshgoftaar, the Pearson Correlation Based Similarity algorithm uses the user's provided rating for a movie as $r_{u,j}$ and the average rating for the same movie across many users as $\bar{r}_j$ to calculate the similarity of items [8].

Su and Khoshgoftaar also proposed implementing a threshold rating as a method to increase our prediction performance [8]. Our approach sets a threshold value at a rating of 3 which limits the ratings we evaluate to 3 or higher and allows our model to focus on all positive movie ratings only for better predictive results. Similar to the Pearson Correlation Based Similarity algorithm, we decided to evaluate our recommendation system by calculating the average of a user's movie ratings at or above a threshold of rating 3 as an 'actual_rating' variable and compares it to the average of all ratings for all recommended movies output by our KNN Model as a 'predicted_rating' variable.

In the evaluation portion of our project, we have used the four different kNN Models representing the Euclidean distance, Cosine Similarity, Minkowski distance, and Manhattan distance metrics that were used in the GUI and applied our modified algorithm. We have decided to use a confusion matrix to compare our models. In doing so, have discovered some limitations of our approach, such as only computing our

confusion matrix values for the true positive and false negative categories since kNN does not have a built-in way to handle negative values. This would make accuracy and recall the main evaluation metrics to compare the models against each other. The results of each model are displayed below (4). Out of the 441 users that had at least one positively rated movie based on our threshold of a rating of 3 or higher, the Euclidean, Manhattan, and Minkowski models performed the best based on their accuracy and recall rates. One surprising result was that the Cosine Similarity Model had a small error rate when it came to predicting the ratings of the users. 4 users were predicted to have a negative rating against our recommendations when in fact they would like the movies we recommended.

IV. CONCLUSION AND FUTURE WORK

Our team has implemented a hybrid movie recommendation system that is able to calculate and give new users a list of popular movies to rate, a User Interface to interact with our hybrid system using a GUI, and once the users have ranked 10 movies or more positively, they will be provided with the results of our four kNN Models. After reviewing the results, three of the four models have produced an excellent accuracy and recall score thus making them highly effective recommendations for users against their positive rated movies.

Even though our models have produced excellent results, there are still opportunities for improvement in the future. A logical future step for this project would be to implement Singular Value Decomposition (SVD) on the larger version of the Movielens dataset. SVD uses linear algebra to express a high dimensional matrix as the product of multiple low dimensional matrices. This can help future projects to speed up their runtime, since it is a technique that can split a large dataset into smaller matrices to compute on.

Another future improvement would be to evaluate different imputation values for unwatched movies, other than inputting zero then ignoring them, and understanding if there is a difference in model performance. One viable way to do this would be to change all unwatched movies to 2.5, which is an average rating from the 0 to 5 range provided.

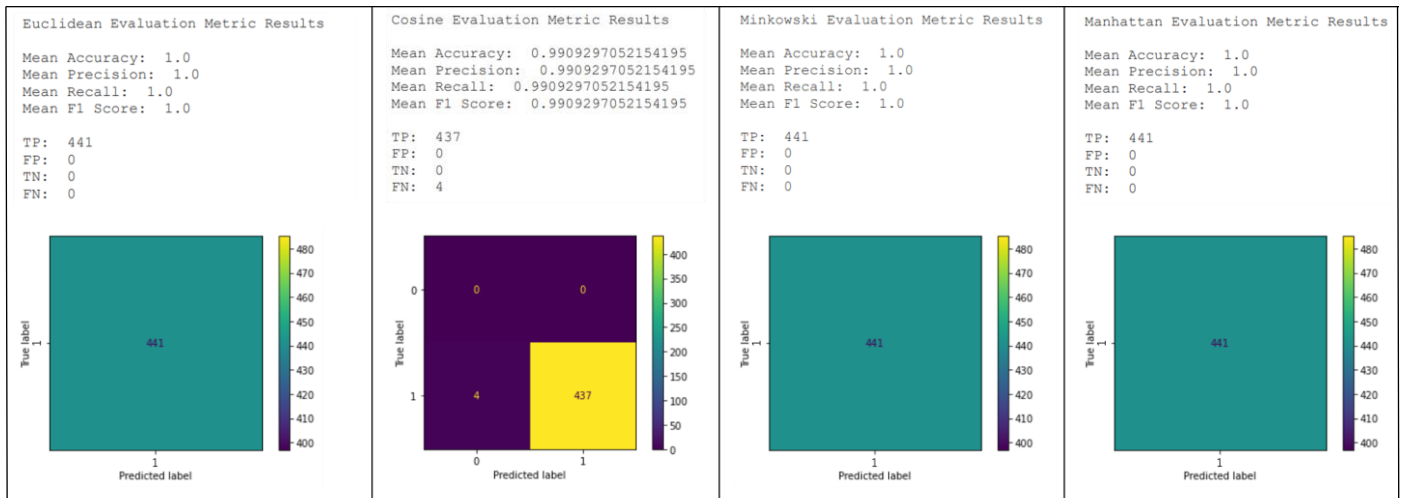


Fig. 4: Model Results

We recommend finding a method to handle negative ratings and evaluate against them, since kNN does not have a built-in way to handle them. The formulas for precision and F1 Score have been provided the code of our project in order to allow others to expand on the confusion matrix once a technique for handling negative ratings is found.

Another opportunity for future work is applying the Cosine Similarity model on a larger dataset, such as the Movielens 25M. Perhaps the Cosine Similarity algorithm would work better on larger datasets over smaller ones, thus explaining why we are seeing a small error rate during our evaluation.

Finally, investigating other rules or popularity-based recommendations that could be used to hybridize cases where kNN falls short, such as before the user has rated any movies or has only rated movies negatively, would continue to be an interesting exercise.

Using a modified Pearson Correlation Based Similarity with a fixed rating threshold algorithm resulted in four kNN models that returned a high accuracy and recall rate overall. The Euclidean, Manhattan, and Minkowski models performed the best when compared against each other's recommendation performance the Cosine Similarity Model.

ACKNOWLEDGMENT

Special thanks to the TSYS College of Computer Science Department.

REFERENCES

- [1] Evelyn Fix, J.L. Hodges. "Nonparametric Discrimination: Consistency Properties"
- [2] Badrul Sarwar, et al. "Application of Dimensionality Reduction in Recommender System - A Case Study"
- [3] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems (TiiS)* 5, 4: 19:1–19:19. <<https://doi.org/10.1145/2827872>>
- [4] Kevin Liao, Prototyping a Recommender System Step by Step Part 1: KNN Item-Based Collaborative Filtering - <https://towardsdatascience.com/prototyping-a-recommender-system-step-by-step-part-1-knn-item-based-collaborative-filtering-637969614ea>
- [5] Das, Palash Ranjan, and Tripti Chakrabarti. "On the Application of Bayesian Credibility Theory to Achieve More Accurate Movie Rankings." *International Journal of Banking, Risk and Insurance* 4.2 (2016): 1-10
- [6] Das, Palash Ranjan, and Tripti Chakrabarti. "On the Application of Bayesian Credibility Theory to Achieve More Accurate Movie Rankings." *International Journal of Banking, Risk and Insurance* 4.2 (2016): 1-10.
- [7] Aiolli, Fabio. (2013). Efficient Top-N Recommendation for Very Large Scale Binary Rated Datasets. *RecSys 2013 - Proceedings of the 7th ACM Conference on Recommender Systems*. 10.1145/2507157.2507189.
- [8] Xiaoyuan Su and Taghi M. Khoshgoftar. "A survey of collaborative filtering techniques." *Advances in Artificial Intelligence* 2009 (2009)
- [9] Saumyadepta Sen, Recommender Systems: Some major recommender system algorithms - <https://medium.com/the-owl/recommender-systems-f62ad843f70c>